AIOps Assignment 1 - Q2

Name: **Vrishab Anurag Venkataraghavan** Roll No.: **DA24B033**

Git Repository Link: `https://github.com/vrishabav/AIOps-A1`

Question 2

2.1

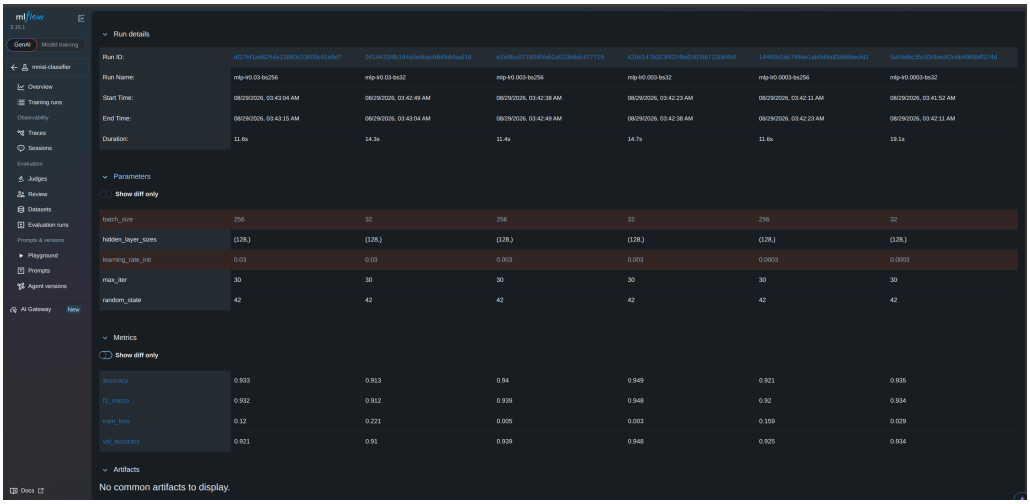

Figure 1: MLFlow run-comparison table

2.2

The best run was mlp-lr0.003-bs32 which had lr=0.003, batch=32, and got a test accuracy of 0.9485 and macro-F1 of 0.9481. Learning rate is in the sweet spot, where it converges within 30 epochs, but remains small enough to be stable (and bs=32 has 8 times more gradient updates per epoch than 256). Both the extremes here failed, where at 0.0003 the training loss was still 0.03-0.16 towards the end with validation accuracy still rising, so those runs were underfitted. For 0.03 it was stationery at 0.12–0.22 i.e. the optimiser was likely overshooting the minimum rather than finding it.

Effectively, most runs showed mild overfitting (apart from lr=0.0003 runs). Validation accuracy typically peaked between epochs 18-26, then flatten or even fall while training loss continued dropping. In the best run, train loss went from 0.466 to 0.0029 while val accuracy peaked at 0.9525 in epoch 21, and becomes 0.9475 (0.005 drop). Among all runs, mlp-lr0.03-bs256 lost/likely overfitted the most, falling 2.87 from its epoch 20 peak.

This all goes to show that learning rate dominates; with fixed batch size, accuracy moves 3.6 for bs = 32, 1.9 for bs = 256. But with constant lr, batch size changes cause differences of around 0.9–2.0. The effect here is that bs 32 ends up beating bs 256 at low learning rates but finally loses at 0.03. This is likely because small batches amplify gradient noise that a large step then works to magnify. Ranking all six runs by lr batch size gives a curve with a single peak, which rises to 0.9485 at 9.4e-5 and falls on either side.

2.3

```python
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("mnist-classifier")

def train_and_log(learning_rate_init=0.001, batch_size=128, hidden_layer_sizes=(128,), max_iter=30, run_name=None):
    with mlflow.start_run(run_name=run_name):
        # --- parameters (at least 3) ---
        mlflow.log_param("learning_rate_init", learning_rate_init)
        mlflow.log_param("batch_size", batch_size)
        mlflow.log_param("hidden_layer_sizes", hidden_layer_sizes)
        mlflow.log_param("max_iter", max_iter)
        mlflow.log_param("random_state", 42)

        model, acc, f1 = train_and_evaluate(learning_rate_init, batch_size, hidden_layer_sizes, max_iter)

        for step, (loss, val_acc) in enumerate(zip(model.loss_curve_, model.validation_scores_)):
            mlflow.log_metric("train_loss", loss, step=step)
            mlflow.log_metric("val_accuracy", val_acc, step=step)

        # --- metrics (at least 2) ---
        mlflow.log_metric("accuracy", acc)
        mlflow.log_metric("f1_macro", f1)

        mlflow.set_tag("team", "data-science")
        mlflow.sklearn.log_model(model, name="model", skops_trusted_types=["sklearn.neural_network._stochastic_optimizers.AdamOptimizer"])

        run_id = mlflow.active_run().info.run_id
        print(f"Logged run {run_id}  |  acc={acc:.4f}  f1={f1:.4f}")
        return run_id
```